

IT644: Web Services & SOA – 100 Practice MSQs

Instructions: Select all options that apply for each question (one or more may be correct). Answers and explanations are provided at the end of this document.

1. A vendor claims their new offering is a “true Web Service.” Which of the following properties, per the standard definition, would you expect it to genuinely exhibit?

- A. It exposes a graphical user interface for end users to click through
- B. It can be invoked by clients regardless of their OS or programming language
- C. It communicates over a network using standard Internet protocols
- D. It requires the client and server to be tightly coupled to the same codebase
- E. It represents a reusable business capability rather than a UI

2. Arrange the evolution of web service communication models in the correct chronological order as presented in the course material.

- A. REST + JSON → SOAP/XML → Microservices → Cloud APIs → AI APIs
- B. SOAP/XML → REST + JSON → Cloud APIs → Microservices → AI APIs → Agentic AI → MCP
- C. Cloud APIs → Microservices → SOAP/XML → REST + JSON → AI APIs
- D. Microservices → AI APIs → SOAP/XML → REST + JSON → Cloud APIs
- E. SOAP/XML → Cloud APIs → REST + JSON → Agentic AI → Microservices

3. A startup is designing a “modern API ecosystem” for its product. Based on the course material, which of the following would genuinely belong in that ecosystem alongside traditional REST APIs?

- A. Punch-card batch processing interfaces
- B. AI Model APIs
- C. Vector Database APIs
- D. Event Streaming APIs
- E. Payment APIs

4. A fintech company wants to secure its public APIs following current best practices. Which combination of measures aligns with the “Security Essentials” covered for web services?

- A. Rely solely on obscurity of the endpoint URL
- B. Use OAuth 2.0 and OpenID Connect for authorization and identity
- C. Apply rate limiting and a Zero Trust posture
- D. Avoid HTTPS/TLS since it adds latency
- E. Use JWTs and API keys for access control

5. A development team wants to document and test their new REST API before release. Which tools/standards, per the course material, are appropriate for this purpose?

- A. OpenAPI (Swagger) for documentation
- B. Postman for testing
- C. AsyncAPI for documenting asynchronous APIs
- D. UDDI as the primary testing framework
- E. Swagger UI for interactive documentation/testing

6. Which statement(s) correctly characterize the shift from the “Traditional Model” to the “Modern Model” of web service interaction as described in the AI-era material?

- A. Traditional model: Application → Web Service; Modern model adds Human↔AI, AI↔AI, AI↔Applications interactions
- B. In the modern model, natural language is increasingly becoming an API interface
- C. The traditional model already included AI↔Enterprise Systems interactions
- D. The modern model eliminates the need for REST APIs entirely
- E. AI agents can interact with cloud services and enterprise systems directly

7. A company is redesigning its infrastructure to be “cloud-native.” Which of the following elements would be part of that architecture per the course material?

- A. Monolithic single-codebase deployment

- B. Kubernetes and Service Mesh
- C. Serverless Computing and Event-Driven Architecture
- D. API Gateway
- E. Manual server provisioning for every new feature

8. A junior developer says, “A web service and a web application are the same thing.” Which statements correctly refute or refine this claim?

- A. A web application has a GUI; a web service typically does not need one, since it’s used as a component
- B. Web services enable machine-to-machine interaction, even across differing tech stacks
- C. Web applications and web services are always mutually exclusive and cannot be part of the same system
- D. Web services are searchable and callable across a network in cloud computing contexts
- E. Every API is automatically a web service

9. A backend engineer is deciding whether to use JSON as the data interchange format for a new multi-language system (Python backend, JavaScript frontend, Java mobile client). Which characteristics of JSON support this choice?

- A. Language-independence, so it isn’t tied to any specific programming language
- B. It is stateful by design, retaining prior request context automatically
- C. It requires all consuming systems to be written in JavaScript
- D. It supports two primary data representations: objects and arrays
- E. It is both human-readable and machine-parsable

10. Which of the following are listed as alternatives to JSON in the course material?

- A. YAML
- B. TOML
- C. MessagePack
- D. WSDL
- E. CBOR

11. A developer states: “JSON is basically a JavaScript object, so it can also contain methods/functions like a JS object.” Evaluate this claim.

- A. Correct — JSON supports embedded functions natively
- B. Incorrect — JSON is similar to JS objects in structure/notation but only has properties, no methods
- C. Correct, because JSON originated from JavaScript syntax
- D. Incorrect — JSON cannot represent nested objects or arrays at all
- E. Partially correct — JSON is similar to JS object literals but is a pure data format

12. In terms of JSON’s core data representations, which correctly matches structure to syntax?

- A. Objects use curly braces { } and represent name-value pairs
- B. Arrays use square brackets [] and represent ordered collections of values
- C. Objects use square brackets and arrays use curly braces
- D. JSON’s common use case includes transmitting data from a server to a web page
- E. JSON is more flexible than HTML for representing complex data structures

13. A legacy banking system needs to expose a web service with formal contracts, strong transactional guarantees, and enterprise-grade security. Which protocol is most appropriate, and why?

- A. REST, because it’s simpler to implement
- B. SOAP, because it supports WS-Security, ACID-compliant transactions, and formal WSDL contracts
- C. SOAP, because it has no learning curve
- D. SOAP, because it natively works in all web browsers without extra libraries
- E. REST, because it inherently supports WS-ReliableMessaging

14. Which of the following are genuine architectural components of a SOAP message?

- A. Envelope — the root element defining the XML document as a SOAP message
- B. Header — optional, can carry metadata like authentication tokens
- C. Body — mandatory, carries the actual request/response data
- D. Cache-Control — mandatory element for browser caching
- E. Fault — optional sub-element of Body providing error information

15. Which statement(s) about SOAP’s historical development and standardization are accurate?

- A. SOAP was created specifically to replace JSON
- B. SOAP 1.2 became a W3C Recommendation in 2003
- C. SOAP was developed by Microsoft, IBM, and others in the late 1990s
- D. SOAP 1.1 was released in 2000, prior to SOAP 1.2
- E. SOAP has been widely adopted in enterprise environments due to extensibility and standards adherence

16. A team is troubleshooting why their SOAP-based service is slower than expected and hard for new developers to onboard onto. Which documented drawbacks of SOAP could explain this?

- A. XML parsing and verbosity increase bandwidth usage and reduce performance
- B. SOAP has a steeper learning curve due to its many specifications
- C. SOAP natively supports multiple lightweight data formats like JSON and YAML
- D. Effective SOAP development often requires specialized tooling/frameworks
- E. SOAP is natively and fully supported by all client-side web browsers

17. Order the following SOAP request-response steps correctly as described in “How SOAP Works.”

- A. Server sends response → Client initiates request → Server processes → Client receives response
- B. None of the above; SOAP does not follow a request-response paradigm
- C. Client receives response → Client initiates request → Server processes → Server sends response
- D. Server processes → Client initiates request → Client receives response → Server sends response
- E. Client initiates request → Server receives and processes → Server sends response → Client receives response

18. Which of the following are part of the WS-Security capabilities that make SOAP suitable for high-security enterprise contexts?

- A. Message Integrity via digital signatures
- B. Automatic GDPR compliance certification
- C. Authentication using tokens like UsernameToken, X.509 certificates, or SAML assertions
- D. Message Confidentiality via encryption
- E. Authorization controlling access based on defined policies

19. A senior architect is defining best practices for a new SOAP-based enterprise service. Which of these align with the documented best practices?

- A. Define clear service contracts using WSDL
- B. Skip versioning since SOAP services never change once published
- C. Use SOAP Faults for robust error handling
- D. Optimize XML payloads to reduce unnecessary elements
- E. Leverage caching where applicable to reduce server load

20. Which key features distinguish SOAP as a protocol, independent of its drawbacks?

- A. Protocol independence — it can run over HTTP, SMTP, or even FTP
- B. It provides built-in support for Web Services Security
- C. It supports stateful operations, where operations can depend on previous ones
- D. It is restricted to running only over HTTP
- E. It cannot support any form of security at all

21. In the SOAP architecture diagram, if a received message’s envelope is intact, what does this imply, and why does it matter?

- A. It implies the message is incomplete and should be discarded
- B. The SOAP envelope packages the entire message, so an intact envelope means the message was completely received and can be parsed for further processing
- C. It means the message used REST instead of SOAP
- D. The envelope is optional and irrelevant to message completeness
- E. It confirms the message can be safely used by the client for further processing

22. A company evaluating SOAP notes it supports “advanced security, transactional reliability, and legacy system integration.” Which of the following scenarios would most justify choosing SOAP over REST, per the “When to Use SOAP” guidance?

- A. A public weather API serving casual mobile app developers
- B. A financial system requiring ACID-compliant, formally-contracted transactions
- C. A scenario requiring integration with an existing legacy SOAP-based system
- D. A simple, cacheable, stateless resource-fetching API for a blog

E. A scenario requiring WS-Security-grade advanced security features

23. Which of the following correctly describe the purpose and nature of WSDL?

- A. WSDL is a binary serialization format for compact network transfer
- B. WSDL specifies service endpoints and message formats to facilitate client-service communication
- C. WSDL is an XML-based language used for describing web services
- D. WSDL was developed by IBM, Microsoft, and others in the early 2000s as part of the SOAP protocol suite
- E. WSDL replaces the need for any service description entirely

24. Match the WSDL structural components to their correct purpose:

- A. Types — defines data types using XML Schema
- B. Messages — describes the data elements of each operation
- C. Port Types — lists the operations supported by the web service
- D. Bindings — provides only the business contact information
- E. Services — provides the endpoint information

25. A developer wants their SOAP client code to be auto-generated instead of hand-written. Which WSDL capability enables this, and what is the underlying mechanism?

- A. WSDL's "Automation" advantage enables tools to generate client code from the WSDL definition
- B. This is impossible; WSDL only documents services for human readers
- C. The generated code is based on the Types, Messages, Port Types, Bindings, and Services defined in WSDL
- D. This capability only works for REST services, not SOAP
- E. WSDL's machine-readable contract is what makes such tooling-based code generation possible

26. Which of the following are documented limitations of WSDL?

- A. It has zero adoption in any enterprise environment today
- B. It creates tight coupling due to strong dependency on SOAP
- C. It can be verbose and difficult to understand (complexity)
- D. There is an increasing preference for simpler RESTful APIs as an alternative
- E. It cannot describe multiple operations for a single service

27. A company is deciding whether WSDL is still relevant for a new project. Based on the "Conclusion and Future of WSDL" material, which statement(s) are accurate?

- A. WSDL plays a crucial role specifically in SOAP-based web services
- B. WSDL has become entirely obsolete and unused in any modern context
- C. The rise of RESTful services may shift how/whether WSDL is used going forward
- D. Understanding both WSDL and REST remains important in modern web service architecture
- E. WSDL is only relevant for describing REST APIs, not SOAP

28. WSDL's relationship to SOAP can best be summarized as which of the following (drawing on comparisons made in the course discussion)?

- A. WSDL is unrelated to SOAP and used exclusively with REST
- B. WSDL is commonly used with SOAP web services and describes how to communicate with them
- C. WSDL plays a role for SOAP analogous to what a .proto file plays for gRPC or a .thrift file plays for Thrift — a machine-readable service contract
- D. A typical SOAP request/response structure can be derived from the service's WSDL definition
- E. WSDL is a runtime execution engine that actually performs the SOAP invocation

29. UDDI is often explained using the "White/Yellow/Green Pages" analogy. Which of the following correctly matches each "page" to its content?

- A. White Pages — business name, description, and contact information
- B. Yellow Pages — categorizes services using taxonomies like NAICS or UNSPSC
- C. Green Pages — technical details for invoking the service (endpoints, protocols, bindings)
- D. White Pages — technical WSDL bindings and invocation details
- E. Yellow Pages — a company's D-U-N-S number and address

30. Which UDDI data model elements correctly correspond to their real-world representation?

- A. BusinessEntity represents a company (White Pages)
- B. BusinessService represents a service provided by a company (Yellow Pages)
- C. BindingTemplate describes how to invoke a service (Green Pages)

- D. tModel defines reusable service standards/protocols for interoperability
- E. tModel represents a company's contact information

31. A company wants to publish and later discover services in a UDDI registry. Which sequence of steps correctly reflects the UDDI publishing and discovery process?

- A. Publishing: define BusinessEntity → publish BusinessService → provide BindingTemplate → define tModel
- B. Discovery: query the registry → search by business/service/tModel → retrieve descriptions and bindings → invoke the service
- C. Publishing requires skipping BindingTemplate entirely since it's optional in all cases
- D. Discovery bypasses the Green Pages and invokes services directly via White Pages contact info
- E. Both publishing and discovery rely on the structured UDDI data model (BusinessEntity, BusinessService, BindingTemplate, tModel)

32. Which of the following are among the four key UDDI APIs described for interacting with a registry?

- A. Inquiry API — for searching services in the registry
- B. Publish API — for publishing business and service information
- C. Rendering API — for generating a GUI for the registry
- D. Security API — for managing access and authentication
- E. Subscription API — for monitoring changes in the registry

33. An enterprise is choosing between a public and a private UDDI registry. Which statement(s) correctly describe this choice?

- A. Public and private registries are functionally identical with no difference in access scope
- B. Private registries are deployed within organizations for internal service discovery and integration
- C. Public registries are accessible by anyone to publish and find services, often used to share info with partners/clients
- D. A private registry would be more appropriate for an organization wanting internal-only service discovery
- E. Public registries cannot use taxonomies like NAICS or UNSPSC

34. Which of the following are documented benefits and challenges of UDDI, respectively?

- A. Benefit: standardized approach to service discovery
- B. Benefit: promotes interoperability and reuse of web services
- C. Challenge: initially slow adoption due to complexity
- D. Challenge: decline in use with the rise of REST APIs and modern alternatives
- E. Challenge: UDDI eliminated all security concerns around open registries

35. A legacy SOA-based enterprise system still uses UDDI for internal service discovery. Which use cases and roles correctly reflect why UDDI would be used in such contexts?

- A. Serving as the actual runtime transport layer for invoking the service (replacing SOAP/REST)
- B. Acting as a distributed registry so services can be discovered in SOA environments
- C. Facilitating B2B integrations by letting companies publish/access third-party services
- D. Enabling cloud service discovery and payment gateway lookups as example use cases
- E. Fully replacing the need for WSDL descriptions of services

36. A system architect must decide between a stateless and stateful design for a new REST-based microservice that just needs to serve independent, self-contained requests. Which characteristics support choosing stateless?

- A. Each request is independent; the server retains no memory of previous interactions
- B. Lower server memory usage since no session data is stored
- C. Harder to scale since session data must be shared across servers
- D. Easier to scale because each request can be distributed across servers without session concerns
- E. The server needs to remember previous interactions to serve the next request correctly

37. An online banking system needs users to stay logged in and navigate multiple pages without re-authenticating. Which architecture is implied, and what trade-off does it bring?

- A. Stateless architecture, because HTTP is inherently stateless and needs no session tracking
- B. This has zero impact on failure recovery complexity compared to stateless designs
- C. This introduces higher server memory usage to maintain each client's session
- D. This introduces scalability challenges, often requiring session persistence or sticky sessions
- E. Stateful architecture, because the server retains session information across requests

38. Which security characteristics are correctly associated with stateless vs. stateful architectures?

- A. Stateless systems often use token-based authentication like JWT, shifting some security responsibility to the client
- B. JWTs used in stateless systems require no protection since they contain no sensitive data
- C. Stateless systems are immune to any form of security compromise
- D. Stateful systems are more vulnerable to attacks like session hijacking or session fixation if not properly managed
- E. Stateful systems place session-data protection responsibility on the server, which can be a centralized-control advantage but also a risk if compromised

39. A gaming company runs a multiplayer server that must track each player's position, health, and inventory in real time. Which resource consumption and scalability implications apply?

- A. This is a stateful use case requiring careful memory and resource management per session
- B. Stateless design would trivially handle this with no downsides
- C. Scaling this system is harder because session data must be shared or maintained per-server
- D. This system would benefit from strategies like session persistence or sticky sessions when scaling
- E. This is an ideal candidate for a purely stateless serverless function architecture

40. A load-balanced set of REST API servers experiences the sudden failure of one instance. Which failure-handling characteristics would apply if the system were stateless?

- A. Failure has minimal impact since no session info is stored on the failed server
- B. The load balancer can route traffic to another instance without needing to recover any session
- C. This would require complex session replication to avoid losing data
- D. This creates server-specific dependencies, making failover difficult
- E. Recovery is comparatively simple, aligning with stateless resilience benefits

41. According to the course material, in which use cases is a stateless approach explicitly recommended?

- A. RESTful services and microservices with independent, idempotent requests
- B. Multiplayer online games requiring persistent player state
- C. Serverless architectures such as AWS Lambda or Google Cloud Functions
- D. Persistent database connections maintaining transactional state
- E. Cloud-based architectures needing easy horizontal scaling

42. A modern web application uses stateless REST APIs on the front end for basic data retrieval, while the back end maintains stateful database connections for authentication and complex transactions. What does this best illustrate?

- A. A pure stateless architecture throughout the system
- B. A pure stateful architecture throughout the system
- C. A hybrid approach combining stateless and stateful components
- D. This combination is explicitly discouraged and never used in modern apps
- E. This reflects how many modern applications actually balance the strengths of both models

43. Which of the following are accurately listed as pros of stateless architecture and cons of stateful architecture, respectively?

- A. Stateless pro: easier fault tolerance and recovery
- B. Stateless pro: simpler development for services not requiring user state
- C. Stateful con: harder to scale due to the need for session persistence
- D. Stateful con: reduced memory and resource usage on the server
- E. Stateless con: every request must include all necessary data (e.g., tokens)

44. A healthcare organization wants multiple systems (patient management, EHR) to reuse a single "patient registration" capability instead of duplicating the logic. Which SOA concept and benefit does this best illustrate?

- A. This illustrates the core SOA idea of using reusable services to provide business capabilities across systems
- B. This is only possible using microservices, not SOA
- C. This reflects the "faster time to market" benefit of SOA over monolithic architectures
- D. This exemplifies "efficient maintenance," since updating the single service doesn't require touching all consuming systems
- E. This is a violation of SOA's core principles

45. Which of the following are the four basic principles of SOA implementation described in the course

material?

- A. Tight coupling — services should have maximum dependency on shared data models
- B. Granularity — each service should ideally pack one discrete business function
- C. Interoperability — services can run regardless of the client's platform/language
- D. Loose coupling — services should have minimal dependency on external resources and be stateless
- E. Abstraction — clients need not know a service's internal implementation details

46. Which of the following correctly describe the four main components of SOA and their sub-features?

- A. Service — has three features: implementation, contract, and interface
- B. Service Provider — creates, maintains, and provides services that others can use
- C. Service Consumer — requests the service provider to run a specific service
- D. Service Registry — a network-accessible directory storing service description documents
- E. Service Registry — the component that actually executes the business logic of a service

47. A company implementing SOA wants to allow multiple communication protocols in one system. Which statement(s) about SOA communication protocols are correct?

- A. Standard protocols to implement SOA include SOAP, RESTful HTTP, Apache Thrift, Apache ActiveMQ, and JMS
- B. Only one protocol can ever be used per SOA implementation
- C. It is acceptable to use more than one protocol within a single SOA implementation
- D. JMS is a specific message broker product, distinct from being a Java API specification
- E. Apache ActiveMQ is one possible implementation used alongside JMS-based messaging

48. An Enterprise Service Bus (ESB) is deployed to connect multiple SOA services. Which of the following correctly describe its role and risk?

- A. An ESB establishes communication between services and consumers regardless of underlying technology
- B. Using an ESB has zero impact on the decentralization goals of SOA
- C. An ESB introduces a single point of failure since it's a centralized service
- D. An ESB provides communication and transformation capabilities through a reusable service interface
- E. If the ESB goes down, clients and services cannot communicate with each other at all

49. Which of the following are documented limitations of SOA?

- A. Limited scalability when services share many resources and must coordinate
- B. Increasing interdependencies that make debugging harder as the system grows complex
- C. SOA has no single point of failure under any configuration
- D. SOA implementations with an ESB introduce a centralized single point of failure
- E. Shared resources like centralized databases can slow down the system

50. How does microservices architecture relate to and differ from SOA, according to the course material?

- A. Microservices retain a centralized ESB just like traditional SOA
- B. Microservices favor data duplication (local data access) as opposed to SOA's tendency toward data sharing
- C. Microservices architecture is described as an evolution of the SOA architectural style
- D. Microservices are fine-grained, independent, and expose functionality through lightweight APIs
- E. Microservices remove the need for a centralized ESB, shifting responsibility to consumers using the API

51. A business analyst says: "SOA services should be like black boxes — consumers shouldn't need to understand their internal code." Which SOA principle(s) does this reflect, and how is it achieved?

- A. This reflects the Abstraction principle
- B. This contradicts the Interoperability principle
- C. This is achieved by giving consumers full access to source code
- D. This is achieved through service contracts and service description documents
- E. This reflects the SOA idea that a service interface reduces dependency between the service and its requester

52. Which statement(s) accurately capture the benefits of SOA over monolithic architectures?

- A. Modifying a service in SOA typically impacts the entire monolithic business process
- B. Efficient maintenance, since small services are easier to create/update/debug than large monolithic code blocks
- C. Faster time to market through reuse of services across business processes
- D. Greater adaptability, allowing modernization (e.g., integrating legacy EHR functionality into newer cloud apps)

E. SOA offers no advantage in reuse compared to monolithic systems

53. A company migrates from a monolith to microservices for an online retail platform. Which changes would correctly reflect this transition per the comparative analysis in the course material?

- A. From a single shared database to a database-per-service model (polyglot persistence)
- B. From independent service deployment back to all-or-nothing deployment
- C. From a tightly coupled unified codebase to loosely coupled, distributed services behind an API Gateway/load balancer
- D. From “scale entire application” to “scale specific services independently”
- E. From small, cross-functional teams to one large, centralized team

54. Which of the following are correctly listed as key characteristics of microservices-based applications?

- A. Decentralized architecture
- B. Great scalability
- C. Centralized shared database as the default design
- D. Failure resistance
- E. Great evolutionary capacity (fast addition of new features)

55. If one microservice in an e-commerce app (e.g., the recommendation service) goes down, which outcome reflects the “failure resistance” characteristic of microservices?

- A. The entire application crashes immediately
- B. The app may lack that specific functionality but continues working otherwise
- C. Users can still browse, add to cart, and even purchase via alternative means if unaffected services remain up
- D. The failed microservice can be isolated/disabled and reconnected after fixing the issue
- E. All other microservices automatically fail as a chain reaction

56. Comparing monolithic and microservices architectures regarding development teams and technology stacks, which statement(s) are accurate?

- A. In a monolith, a single team of engineers typically develops, tests, and maintains the app
- B. In microservices, separate teams are typically allocated per service
- C. Monoliths allow each feature to use a completely different technology stack freely
- D. Microservices allow each service to be developed using different technologies
- E. Both architectures enforce an identical, unified tech stack across all components

57. Which problems does the microservice architecture help solve for large, fast-growing businesses, according to the course material?

- A. Complex, hard-to-maintain codebases from constantly evolving monoliths
- B. Poor scalability due to needing to scale the entire application at once
- C. Guaranteed elimination of all bugs across the system
- D. Unacceptable downtime, by avoiding full-app redeployment for every change
- E. Deficient technology stack capabilities, by allowing different tech per service

58. In the “Monolith at Scale vs Microservices” comparison, which solution correctly pairs with its stated problem?

- A. Problem: Large single codebase → Solution: Independent codebases maintained by dedicated teams
- B. Problem: Outdated technologies → Solution: Individual tech stacks per microservice
- C. Problem: Slow development/testing → Solution: A single autonomous team develops everything together
- D. Problem: More bugs from complex architecture → Solution: Efficient, per-service testing
- E. Problem: Complete system scaling → Solution: Independent scaling per service based on unique performance needs

59. Regarding “time to market,” which statement correctly reflects the graph comparing monolith vs. microservices as solution complexity increases?

- A. Monolith’s time-to-market stays low and flat regardless of complexity
- B. Microservices’ time-to-market rises sharply and unpredictably as complexity increases
- C. Monolith’s time-to-market rises sharply as solution complexity increases
- D. At low complexity, monoliths can actually be faster/easier to launch initially
- E. Microservices show a more gradual, controlled increase in time-to-market as complexity grows

60. Which of the following statements about the microservices architectural model (App → Microservice 1/2/3 → separate Databases 1/2/3) are accurate?

- A. Each microservice generally has its own dedicated database
- B. All microservices in the model must share exactly one central database
- C. The app layer routes to independent microservices, each handling its own function and data
- D. This model enables independent scaling of each microservice/database pair
- E. This model requires every microservice to use the same programming language

61. According to the “Key Characteristics of Microservice Apps,” which pairing of characteristic and explanation is correct?

- A. Independent components — distinctive microservices don’t affect other components of an application
- B. Failure resistance — if a microservice fails, the whole application must be taken offline for safety
- C. Great evolutionary capacity — applications can quickly grow and gain new functionality by adding features fast
- D. Decentralized approach — no need to maintain an expensive centralized database/server for all information
- E. Decentralized approach — this restricts the ability to apply various technologies

62. An online store owner wants to connect independently maintained tools (accounting system, payment gateway, inventory management, customer portal, web analytics) via API/HTTP integrations. Which microservices benefit does this best illustrate?

- A. Simplified Onboarding
- B. High Sustainability
- C. Self-sufficiency — each service/tool works independently and can be maintained separately
- D. Continuous Delivery
- E. This illustrates a monolithic design pattern instead

63. A company wants to use Python for its ML-based recommendation service while keeping its main web app in Java. Which microservices benefit supports this choice, and what does the course material say about language selection?

- A. Technical variety — microservices allow different languages/technologies per service
- B. Any programming language can be used to develop microservices to leverage its strengths
- C. Python is specifically called out as commonly used for machine learning and AI
- D. This is impossible in a microservices architecture, which enforces one unified stack
- E. Java, GoLang are noted as suited for web/mobile applications and embedded systems

64. Netflix’s web application uses more than 700 microservices, each independently scaled. Which benefit(s) does this best demonstrate?

- A. Scalability — a distinctive technology/database per service can be scaled independently
- B. There is a hard limit on the number of microservices a system can use
- C. This demonstrates high sustainability only, not scalability
- D. Businesses can start small and expand resources for specific services as load increases
- E. New microservices/functionality can be added to expand what a company offers

65. A payment gateway integrated into a checkout system suddenly stops working, but users can still browse, add to cart, and check out using an alternative payment method. Which benefit is illustrated?

- A. Simplified Onboarding
- B. Continuous Delivery
- C. Technical Variety
- D. This demonstrates the ability to allocate and fix errors as a top benefit of microservices
- E. High Sustainability — the app’s functionality is poorly affected since other microservices remain unaffected by one node’s outage

66. Which of the following correctly describe “Continuous Delivery” and “Easy Changes Implementation” as microservices benefits?

- A. Continuous Delivery — developers can upgrade certain services without disabling the entire application
- B. Easy Changes Implementation — minor updates require redeploying the entire system
- C. Easy Changes Implementation — a microservice can be temporarily disabled for minor changes without affecting the whole app
- D. Continuous Delivery — CI/CD environments are harder to establish with microservices than monoliths
- E. Both benefits support releasing updates while keeping maximum application uptime

67. According to the Microsoft survey referenced in the course material, which application types rank among

the top five microservices use cases, and in what approximate order?

- A. Data analytics (45%) ranks highest among the cited use cases
- B. Database applications (41%) rank second
- C. Customer relationship management (38%) ranks third
- D. HR applications (31%) rank ahead of Finance (34%)
- E. Finance (34%) ranks fourth, ahead of HR applications (31%)

68. Based on the microservices implementation statistics cited, which of the following are accurate?

- A. Roughly 81.5% of companies already use microservices
- B. About 92% of businesses reported a successful switch to microservices architecture
- C. Only a small minority (under 50%) of architecture adopters plan to keep investing in microservices development
- D. 87% of business owners stated microservices adoption expenses were worth it
- E. 1% of companies indicated they will not adopt microservices at all

69. A business is deciding whether to adopt microservices. Based on the “Ifs and Buts” guidance, which condition(s) support choosing microservices architecture?

- A. The company expects high-pace growth and wants to modernize its application quickly
- B. The company is fully satisfied with current product development time and needs no scaling
- C. The company can afford to employ several teams of developers
- D. The company strives to develop a highly reliable, failure-proof application
- E. The company wants to use various technologies across its stack

70. A team is modernizing a decade-old, hard-to-maintain application built on outdated technology, without breaking the whole system at once. Which microservices use case and approach applies?

- A. “Outdated Applications” use case — break the monolith into microservices and apply new tech to separated parts
- B. This is a “Data-Heavy Applications” use case exclusively
- C. This aligns with the strangler pattern of extracting features incrementally
- D. This use case requires a complete, immediate rewrite with no incremental option
- E. New technologies can be applied to just the separated/extracted parts, not the whole system at once

71. A healthcare monitoring platform must maintain near-100% uptime and process data reliably even if some components fail. Which use case and characteristic combination applies?

- A. “Highly-Resilient Applications” use case
- B. Even if one or more microservices are down, the entire application can keep working, providing partial functionality
- C. This use case is only relevant to gaming applications
- D. “Apps that Process Real-Time Data” is irrelevant to healthcare monitoring
- E. Microservices help build highly resilient and secure apps for industries like healthcare/finance

72. A social media platform with many interacting features (feeds, messaging, notifications, search) is a candidate for which use case, and why?

- A. “Apps with Complex Logic” use case — many components in the architecture benefit from distinct features using different technologies
- B. This should instead be built as a single monolithic codebase for simplicity
- C. Microservices allow developing many varied features and connecting them
- D. This use case has nothing to do with e-commerce-style applications
- E. Complex-logic apps are explicitly named as a foremost microservices use case

73. An analytics platform ingests large volumes of data from many different sources with heavy interdependencies. Which use case applies, and what does microservices architecture offer here?

- A. “Data-Heavy Applications” use case
- B. This scenario should avoid microservices entirely due to data volume
- C. This use case is unrelated to “Apps that Process Real-Time Data”
- D. Real-time data volume needs can be allocated with the required storage/computing power on demand
- E. Microservices help use resources wisely, reducing architectural complexity from many dependencies

74. A team is choosing between “All-in-One Auth Service,” “Shared Session Storage,” and “JWT” for microservices authentication. Which pros/cons pairings are accurate?

- A. All-in-One Auth Service: great encapsulation and always up-to-date basic user info, but high load/bottleneck risk on the auth service
- B. Shared Session Storage: easy to implement with no strict dependencies between auth and resource services, but poor encapsulation
- C. JWT: no performance bottleneck or single point of failure, but more data transferred over the network and difficulty handling logouts
- D. JWT is described as the simplest option requiring no senior developer expertise
- E. All-in-One Auth Service creates strict dependency of resource services on the authentication service

75. Comparing monolithic vs. microservices authentication/authorization approaches, which statement(s) are correct?

- A. Monolithic: typically uses one database and one backend app issues/verifies ID tokens
- B. Both architectures use an identical, single shared login and token verification mechanism
- C. Monolithic: all additional user data is stored in the same database
- D. Microservices: additional user info and data are typically stored in different databases across services
- E. Microservices: a dedicated login service is required, and different services verify tokens independently

76. A fintech company wants a “ready-to-use” Customer Identity and Access Management (CIAM) solution instead of building authentication from scratch. Which platforms and matching features are correctly cited?

- A. AWS Cognito — supports self-registration, MFA, and machine-to-machine authentication
- B. UDDI — is a leading CIAM platform for microservices authentication
- C. Auth0 — supports social login, passwordless authentication, and real-time breach detection
- D. FusionAuth — supports single sign-on, biometric authentication, and IoT authorization
- E. All three CIAM platforms (Cognito, FusionAuth, Auth0) are explicitly named as widely used

77. Which best practices are recommended for securing communication between microservices?

- A. Use HTTPS with services validating certificates of the services they connect to
- B. Rely exclusively on network obscurity instead of encryption
- C. Use a service mesh to simplify communication, detect errors, and encrypt/validate data
- D. Service mesh implementations typically use sidecars that work as proxies
- E. Avoid streamlining development/testing/deployment when using a service mesh

78. Which of the following correctly reflect the “Microservices Security Implementation - Best Practices” checklist?

- A. Authentication and authorization — a solution enabling effective auth for every microservice (custom-built or ready-to-use)
- B. Secured data storage — establishing in-transit and at-rest encryption, managing data lifecycle and automatic backups
- C. Sensitive data management — hardcoding login credentials for simplicity and speed
- D. API gateway + Firewall — configuring the combination to secure access to a microservices-based application
- E. Secured communication — ensuring data travels securely between microservices using encryption and additional security layers

79. Why are microservices considered inherently more challenging to secure than monolithic applications, per the “Microservices Security Challenges” material?

- A. Monolithic and microservices security challenges are identical since both use one shared database
- B. Many decoupled services need to communicate and share sensitive information, increasing exposure points
- C. Every microservice may use a dedicated database and diverse technologies, complicating unified authentication/authorization
- D. Microservices security requires a more complex solution than the single centralized approach used in monoliths
- E. There is no need for developers to think differently about auth in microservices vs monoliths

80. A fintech app needs “the most robust solution” for securing communication between its Credit Check and Transaction Processing microservices. Based on the course material, which combination is most appropriate?

- A. Using shared session storage exclusively, regardless of encapsulation concerns
- B. Relying only on a custom-built authentication solution for every service, built from scratch
- C. Establishing in-transit data encryption using HTTPS and utilizing a service mesh for validation
- D. Avoiding hardcoded credentials and configuring an API Gateway
- E. Applying least privilege and implementing JSON Web Tokens (JWT)

81. Which statement(s) correctly compare “shared session storage” against “JWT” for microservices authentication?

- A. Shared session storage offers great performance but at the cost of some microservices principles (like independence/decoupling)
- B. JWT promotes a decentralized approach since basic user info can be read directly from the token
- C. Shared session storage has no risk of becoming a performance bottleneck
- D. JWT requires more data to be transferred over the network compared to simple ID lookups
- E. Shared session storage eliminates strict dependencies between authentication and resource services, unlike the all-in-one approach

82. A company is unhappy with its monolith’s delivery speed and is considering microservices. Per the migration guide’s Step 1 (“Project problems source identification”), which underlying issues should first be ruled out before blaming the architecture?

- A. The use of a shared database (this is always the root cause, per Step 1)
- B. Poor team management
- C. Insufficient business analysis
- D. Unoptimized development processes
- E. Lack of testing

83. After ruling out non-architectural causes, a company still believes microservices is the right move. What should happen next, according to Steps 2–4 of the migration guide?

- A. Step 4: immediately perform a big-bang rewrite with no incremental option considered
- B. Step 3: optimize the monolith following DDD/Clean Architecture guidelines to enable smooth refactoring
- C. Step 2: analyze whether issues can be solved without converting to microservices at all
- D. Step 4: choose the best refactoring approach between a complete and an incremental rewrite
- E. Skip monolith optimization entirely and extract microservices directly from unoptimized code

84. Which of the following correctly describe Steps 5–7 of the monolith-to-microservices migration guide?

- A. Step 5: prioritize features to extract based on whether they’ll accelerate development, solve project issues, or help extract other services
- B. Step 6: create a strangler application by extracting important features and connecting them to the monolith
- C. Step 7: develop new features as distinctive microservices connected to the monolith
- D. Step 6 requires deleting the monolith entirely before any extraction begins
- E. Step 5 has no defined prioritization criteria — extraction order is arbitrary

85. A team extracts several “microservices” from their monolith, but these new services remain highly connected to and dependent on the original monolith, so the original complexity issues persist. What outcome does this describe, and why is it problematic?

- A. This is the ideal, desired outcome of any microservices migration
- B. It means the newly extracted services are highly connected/heavily dependent on the monolith
- C. This describes a “Distributed Monolith” — an undesirable migration outcome
- D. “Monolith issues at scale” remain unsolved even though the app is technically split into services
- E. This outcome has no bearing on whether the migration was successful

86. Visually, how does a “Distributed Monolith” differ from proper “Microservices,” based on the comparison diagrams (tangled lines vs. clustered connections)?

- A. Distributed Monolith shows many crossing, tangled dependencies across all components
- B. Microservices shows dependencies clustered within logical service groups, with fewer cross-cutting connections
- C. Both diagrams show identical dependency patterns
- D. The Microservices diagram indicates better separation of concerns despite still having some necessary inter-service links
- E. The Distributed Monolith diagram indicates a lack of clear service boundaries

87. A team wants to avoid ending up with a “Distributed Monolith” during their migration. Which practices from the 7-step guide would most directly help avoid this outcome?

- A. Properly identifying whether the root problems are architectural before migrating (Step 1–2)
- B. Skipping monolith optimization to save time (opposite of Step 3)
- C. Carefully choosing between complete and incremental rewrite approaches rather than rushing (Step 4)

- D. Extracting features into a well-defined strangler application rather than loosely bolting services onto the monolith (Step 6)
- E. Ignoring feature prioritization criteria and extracting features randomly

88. Amazon's 2001 codebase crisis involved developers spending excessive time resolving merge conflicts across a large, complex codebase. Which statement(s) accurately describe Amazon's response and outcome?

- A. Amazon avoided any automation efforts and continued manual testing indefinitely
- B. Developers pulled out independent microservices and connected them using API integrations
- C. Amazon decomposed the monolith by analyzing the codebase and allocating functional units serving a single purpose
- D. Amazon today reportedly uses over 1000 microservices
- E. Amazon engineers developed automation solutions to eliminate idle time in their CI/CD pipeline

89. Netflix's transition to microservices was triggered by which specific problem, and what was the broader strategic shift involved?

- A. Netflix decided to keep its on-premises database rather than move to AWS
- B. A shift from continuous vertical scaling (single point of failure risk) to horizontal scaling via the cloud
- C. Processing challenges leading to database failures as popularity grew
- D. Developers had to decouple the monolith to move the database to the cloud
- E. Netflix now reportedly uses over 700 microservices

90. Uber's original architecture and subsequent shift to microservices are best described by which statement(s)?

- A. Uber launched as a monolith handling driver-passenger matching, payments, and trip management in one codebase
- B. Even minor changes required comprehensive testing since they affected the entire codebase
- C. Microservices helped Uber scale billing, trip management, driver management, and notifications independently
- D. Uber's monolithic architecture made it trivial to release new features quickly at scale
- E. Uber reportedly uses over 500 microservices today

91. Which company-specific detail correctly matches the real-world use case described in the course material?

- A. Amazon — over 1000 microservices
- B. Netflix — over 700 microservices
- C. Uber — over 500 microservices
- D. Uber was the "youngest business case for microservices adoption" among the three
- E. Netflix is described as the "youngest business case for microservices adoption" among the three

92. Across all three real-world case studies (Amazon, Netflix, Uber), which common theme(s) explain why each company moved toward microservices?

- A. Each company started as (or evolved into) a large monolith that became difficult to scale or maintain as it grew rapidly
- B. Each company needed to release/update features faster without risking the entire codebase
- C. None of the three companies faced any downtime or reliability issues before migrating
- D. All three companies ultimately adopted independently deployable, independently scalable services
- E. Rapid business growth was a key driver for reconsidering the monolithic approach in each case

93. Which statement(s) accurately describe the three main types of cloud computing service models as covered in the AWS case study material?

- A. IaaS provides the basic building blocks of cloud IT: networking, compute, and storage, giving the highest flexibility/control
- B. PaaS removes the need to manage underlying infrastructure, letting you focus on deploying/managing your applications
- C. SaaS requires you to manage the OS and infrastructure yourself while the vendor only provides the application code
- D. Each service model provides different levels of control, flexibility, and management
- E. SaaS delivers a complete product managed entirely by the service provider, typically accessed via subscription

94. According to the "Benefits of AWS" material, which of the following are explicitly cited advantages of AWS?

- A. Highest cost regardless of usage, due to fixed pricing with no flexibility
- B. Largest community — AWS Partner Network (APN) includes thousands of systems integrators and tens of thousands of ISVs
- C. Most functionality — more services and deeper feature sets than any other provider
- D. Fastest pace of innovation — e.g., pioneering serverless computing with AWS Lambda in 2014
- E. Elasticity — scaling resources up/down instantly instead of over-provisioning upfront

95. A systems integrator formally partners with AWS to get technical support and co-marketing benefits, while also being a company that builds and sells its own software product. Which terms correctly apply, and how do they relate?

- A. The company qualifies as an Independent Software Vendor (ISV) because it builds and sells software
- B. By formally partnering with AWS, it becomes part of the AWS Partner Network (APN)
- C. ISV and APN Partner are mutually exclusive categories — a company cannot be both
- D. The APN includes thousands of systems integrators and tens of thousands of ISVs, per the AWS benefits material
- E. Being an APN partner automatically means the company no longer sells its own software

96. Which historical facts about AWS's origin and early growth are accurate per the case study material?

- A. A 2003 paper by Benjamin Black and Chris Pinkham envisioned standardized, automated infrastructure relying on web services
- B. AWS's first cloud computing service launched was Simple Storage Service (S3) in March 2006
- C. AWS was created purely because Amazon had unused "excess capacity" from its e-commerce site, per verified fact
- D. Netflix became the first company to publicly announce running all its infrastructure on AWS, in 2010
- E. EC2 and SQS followed S3 into production shortly after its 2006 launch

97. Comparing major cloud providers per the case study, which statement(s) are accurate?

- A. AWS is described as the market leader by overall market share and offering breadth
- B. Microsoft Azure is generally ranked second, with deep strength in enterprise IT and hybrid cloud
- C. Google Cloud Platform (GCP) is known for strength in data analytics/AI, including TensorFlow and Kubernetes
- D. IBM Cloud's advantage includes its acquisition of RedHat, enabling use of the OpenShift PaaS platform
- E. Oracle Cloud Infrastructure (OCI) is primarily favored by startups and small businesses over large enterprises

98. Which of the following is cited as a genuine weakness or competitive risk for AWS, according to the case study?

- A. AWS lacks strong offerings for hybrid cloud deployments compared to Azure and IBM
- B. Enterprises already using Microsoft/IBM products may gravitate to those providers for hybrid cloud needs
- C. AWS has guaranteed permanent market leadership with no risk of being overtaken
- D. AWS must continue innovating in AI, ML, IoT, and containers to retain its market position
- E. Technology markets can shift quickly, so current leadership doesn't guarantee future performance

99. Match each AWS customer case study to its correctly described use/benefit:

- A. Adobe — primarily uses AWS solely for marketing website hosting, unrelated to its Creative Cloud products
- B. Siemens — built MindSphere, an IoT operating platform hosted on AWS, to enable Industry 4.0 transformation
- C. Netflix — took 7 years to fully migrate to AWS, moving from a single application to thousands of microservices
- D. Hotstar (Novi Digital) — used AWS for cost-effective scalability, receiving up to 5700 Gbps of bandwidth during a live match
- E. BMW — built its CARASSO connected-car sensor service in six months using AWS services like S3, SQS, DynamoDB, RDS, and Elastic Beanstalk

100. A media company wants to minimize latency for a performance-sensitive workload (e.g., video editing in the cloud) using AWS, drawing on the Adobe case study. Which practices/facts are relevant?

- A. Being physically closer to an AWS region (e.g., via AWS Local Zones) improves user experience by reducing latency
- B. AWS Direct Connect can enhance connectivity through dedicated capacity, adding a security layer by bypassing the public internet path

- C. A target latency of around 30ms or less is cited as providing an optimal experience
- D. Higher latency has no impact on peripheral device interactivity such as playback or graphical interface responsiveness
- E. Different user personas (e.g., News/Sports, Creative, Promo) have different demands for storage, bandwidth, disk I/O, CPU, and memory

Answers & Explanations

- 1. Answer:** B, C, E **Explanation:** A web service is platform/language-independent, standards-based, network-accessible, loosely coupled, and reusable — it exposes business capabilities, not user interfaces (A is wrong; D contradicts loose coupling).
- 2. Answer:** B **Explanation:** The documented evolution is SOAP/XML → REST/JSON → Cloud APIs → Microservices → AI APIs → Agentic AI → Model Context Protocol (MCP), reflecting a shift from app-to-app to AI-to-AI interaction.
- 3. Answer:** B, C, D, E **Explanation:** The modern API ecosystem listed includes REST, GraphQL, gRPC, AI Model APIs, Vector Database APIs, Search, Authentication, Payment, Messaging, and Event Streaming APIs. Punch-card interfaces are not part of this list (and are anachronistic).
- 4. Answer:** B, C, E **Explanation:** Security essentials covered are HTTPS/TLS, OAuth 2.0, OpenID Connect, JWT, API Keys, Rate Limiting, Zero Trust, and Identity Federation. Relying on obscurity or skipping HTTPS contradicts these standards.
- 5. Answer:** A, B, C, E **Explanation:** Documentation tools include OpenAPI/Swagger, AsyncAPI, and API Portals; testing tools include Postman, Swagger UI, and unit/integration testing. UDDI is a service registry/discovery mechanism, not a testing framework.
- 6. Answer:** A, B, E **Explanation:** The traditional model was simple application-to-service; the modern model expands to Human↔AI, AI↔AI, AI↔Applications, AI↔Cloud Services, and AI↔Enterprise Systems, with natural language becoming an API interface. REST APIs still exist as a core technology (D is false); traditional model did not include AI interactions (C is false).
- 7. Answer:** B, C, D **Explanation:** Cloud-native architecture includes Microservices, Containers (Docker), Kubernetes, Service Mesh, API Gateway, Serverless Computing, Event-Driven Architecture, and Cloud-Native Applications — the opposite of monolithic, manually-provisioned setups.
- 8. Answer:** A, B, D **Explanation:** Web apps provide a GUI for humans; web services expose capabilities to other programs, often without a UI, enabling machine-to-machine interoperability. They frequently coexist (a web app’s backend may call web services), so C is false. Not every API is a web service — only those using web protocols over a network (E is false). ## Section B: JSON (Q9–Q12)
- 9. Answer:** A, D, E **Explanation:** JSON is human-readable, machine-parsable, and language-independent, using objects (name-value pairs in braces) and arrays (ordered values in brackets). It is not tied to JavaScript despite the name, and JSON itself carries no built-in statefulness.
- 10. Answer:** A, B, C, E **Explanation:** Listed alternatives are YAML, TOML, XML, CSV, MessagePack, HJSON, CBOR, and YAML 1.2. WSDL is a service description language, not a data interchange format alternative to JSON.
- 11. Answer:** B, E **Explanation:** JSON is similar to JavaScript objects in how it’s written/stored, but it contains only properties (data), not methods (behavior). It absolutely can represent nested objects/arrays (D is false).
- 12. Answer:** A, B, D, E **Explanation:** Objects = { } (name-value pairs); Arrays = [] (ordered values). JSON is commonly used for server-to-web-page data transfer and is described as more flexible than HTML for complex data. ## Section C: SOAP (Q13–Q22)
- 13. Answer:** B **Explanation:** SOAP is recommended when formal WSDL contracts, WS-Security, and ACID transactional reliability are required — exactly the banking scenario. SOAP has a steep learning curve and limited browser support, and WS-ReliableMessaging is a SOAP-specific feature, not REST’s.
- 14. Answer:** A, B, C, E **Explanation:** SOAP’s architecture includes Envelope, Header (optional), Body (mandatory), Fault (optional, inside Body), and Attachment (for binary data). “Cache-Control” is an HTTP header concept, not a SOAP message component.
- 15. Answer:** B, C, D, E **Explanation:** SOAP originated in the late 1990s (Microsoft, IBM, others), with SOAP

1.1 (2000) followed by SOAP 1.2 (2003, W3C Recommendation). JSON postdates and is unrelated to SOAP's original purpose (A is false).

16. Answer: A, B, D **Explanation:** Documented SOAP drawbacks include performance overhead from XML verbosity, a steep learning curve, tooling dependence, and limited browser support (E is false) and limited flexibility in data formats — it's primarily XML-based, not multi-format (C is false).

17. Answer: E **Explanation:** SOAP operates via request-response: the client constructs and sends a SOAP request, the server parses/processes it, constructs and sends a response, and the client parses and uses the response.

18. Answer: A, C, D, E **Explanation:** WS-Security covers message integrity (digital signatures), confidentiality (encryption), authentication (UsernameToken/X.509/SAML), and authorization (policy-based access control). It does not automatically grant any specific legal compliance certification.

19. Answer: A, C, D, E **Explanation:** Best practices include using WSDL for contracts, implementing strong security, maintaining versioning (not skipping it — B is false), optimizing XML payloads, SOAP Fault-based error handling, caching, and documentation.

20. Answer: A, B, C **Explanation:** SOAP's key features are protocol independence (HTTP/SMTP/FTP), built-in WS-Security support, and support for stateful operations. It is explicitly not restricted to HTTP alone, and it does support security extensively.

21. Answer: B, E **Explanation:** The envelope's role is to package the whole SOAP message; if it's present and complete, the client can parse and use the message, confirming successful, complete transmission.

22. Answer: B, C, E **Explanation:** SOAP is recommended for formal WSDL contracts, WS-Security needs, ACID-compliant transactions, and legacy SOAP system integration — not for simple, cacheable, public-facing APIs where REST's simplicity wins (A, D favor REST). ## Section D: WSDL (Q23-Q28)

23. Answer: B, C, D **Explanation:** WSDL is an XML-based description language (not binary) developed alongside SOAP in the early 2000s by IBM, Microsoft, and others, specifying endpoints and message formats — it IS the service description, not a replacement for having one.

24. Answer: A, B, C, E **Explanation:** WSDL's five main components are Types (data types via XML Schema), Messages (data elements per operation), Port Types (operations offered), Bindings (protocol and data format per operation — not contact info), and Services (endpoint info).

25. Answer: A, C, E **Explanation:** WSDL's advantage of "Automation" allows tools to generate client-side code by reading the machine-readable contract (Types/Messages/PortTypes/Bindings/Services) — this is specifically a SOAP/WSDL capability, not a REST one.

26. Answer: B, C, D **Explanation:** WSDL's limitations include complexity/verbosity, tight coupling to SOAP, and REST's growing popularity as a simpler alternative. It still sees enterprise use (A is false) and can describe multiple operations via Port Types (E is false).

27. Answer: A, C, D **Explanation:** WSDL remains crucial for SOAP-based services and still sees use in certain (often enterprise/legacy) contexts, even as REST's rise changes usage dynamics — it isn't obsolete, but understanding both remains valuable. WSDL is tied to SOAP, not REST (E is false).

28. Answer: B, C, D **Explanation:** WSDL is commonly paired with SOAP, describing its communication method and enabling typical SOAP request/response structuring — conceptually similar to schema files used by gRPC/Thrift for code generation. WSDL is a description/contract, not an execution engine (E is false), and it is SOAP-related, not REST-exclusive (A is false). ## Section E: UDDI (Q29-Q35)

29. Answer: A, B, C **Explanation:** White Pages = business/contact info; Yellow Pages = service categorization via taxonomies (NAICS/UNSPSC); Green Pages = technical invocation details (endpoints, protocols, WSDL bindings). D and E swap the categories incorrectly.

30. Answer: A, B, C, D **Explanation:** UDDI's core data structures are BusinessEntity (company/White Pages), BusinessService (service/Yellow Pages), BindingTemplate (invocation details/Green Pages), and tModel (technical standards/protocols for interoperability — not contact info).

31. Answer: A, B, E **Explanation:** The documented process: publishing involves BusinessEntity → BusinessService → BindingTemplate → tModel; discovery involves querying, searching, retrieving, and invoking via Green Pages (binding) details — not via White Pages contact info.

32. Answer: A, B, D, E **Explanation:** The four key UDDI APIs listed are Inquiry, Publish, Security, and Subscription. There is no "Rendering API" in the UDDI specification as covered.

33. Answer: B, C, D **Explanation:** Public registries are open for external publishing/discovery (e.g., with partners), while private registries serve internal organizational needs — these are meaningfully different in scope (A is false). Yellow Pages taxonomies like NAICS/UNSPSC apply regardless of registry type (E is false).

34. Answer: A, B, C, D **Explanation:** Benefits include standardization, interoperability/reuse, simplified integration, and reduced service-discovery time. Challenges include slow initial adoption, decline due to REST's rise, security concerns with open registries (not “eliminated” — E is false), and competing discovery methods like DNS-based services.

35. Answer: B, C, D **Explanation:** UDDI's role is discovery/registry (SOA environments, B2B integration, cloud service/payment gateway discovery) — it works alongside WSDL and SOAP/REST rather than replacing them as a transport or description mechanism (A, E are false). ## Section F: Stateful vs. Stateless Architecture (Q36–Q43)

36. Answer: A, B, D **Explanation:** Statelessness means independent requests, lower memory usage, and easier horizontal scaling since there's no session to synchronize. C and E describe stateful characteristics instead.

37. Answer: C, D, E **Explanation:** Persistent login/navigation without re-authentication requires stateful session tracking, which increases server memory usage and complicates scaling (needing sticky sessions/session persistence) and failure recovery (B is false — stateful failure recovery is more complex).

38. Answer: A, D, E **Explanation:** Stateless architectures shift security to client-held tokens (e.g., JWT) which must still be protected from hijacking (B is false); stateful architectures centralize session data on the server, which is convenient but risks session hijacking/fixation if compromised. No system is “immune” to compromise (C is false).

39. Answer: A, C, D **Explanation:** Real-time multiplayer state tracking is inherently stateful, requiring memory/resource management and scaling strategies like sticky sessions or persistence — it is not a natural fit for stateless/serverless design (B, E are false).

40. Answer: A, B, E **Explanation:** Stateless systems are resilient to failure since no session state needs recovering — traffic reroutes easily. C and D describe stateful failure-handling challenges, not stateless ones.

41. Answer: A, C, E **Explanation:** Stateless architecture fits simple, idempotent requests (APIs, microservices), serverless functions, and cloud-based horizontal scaling. Multiplayer games and persistent DB connections are listed as stateful use cases (B, D).

42. Answer: C, E **Explanation:** This is a textbook hybrid approach — stateless front-end services paired with stateful back-end services — explicitly described as common in modern applications, not discouraged.

43. Answer: A, B, C, E **Explanation:** Stateless pros include easier scaling/fault tolerance, lower resource usage, and simpler development; its con is that every request needs full data (tokens etc.). Stateful cons include harder scaling and higher resource usage (D incorrectly states reduced usage — that's a stateless pro, not a stateful con). ## Section G: Service-Oriented Architecture — SOA (Q44–Q52)

44. Answer: A, C, D **Explanation:** This is a direct example of SOA: reusing a business-capability service (e.g., patient registration) across multiple systems, illustrating both faster development (reuse instead of rewriting) and efficient maintenance (updating one service, not many). This is a foundational SOA pattern, not exclusive to microservices, and not a violation.

45. Answer: B, C, D, E **Explanation:** SOA's basic principles are Interoperability, Granularity, Loose Coupling (not tight coupling — A is false and contradicts loose coupling), and Abstraction (services as “black boxes”).

46. Answer: A, B, C, D **Explanation:** SOA's four components are Service (with implementation, contract, interface), Service Provider, Service Consumer, and Service Registry (a discovery directory, not an execution engine — E is false).

47. Answer: A, C, E **Explanation:** SOA protocols include SOAP, REST, Thrift, ActiveMQ, and JMS, and multiple protocols can coexist in one SOA implementation (B is false). JMS is a Java messaging API specification (not a product itself — D reverses this); ActiveMQ is an implementation that can support JMS-style messaging.

48. Answer: A, C, D, E **Explanation:** An ESB acts as a centralized router/transformer between services (A, D), but this centralization directly contradicts SOA's decentralization ideal (B is false) and creates a single point of failure — if it goes down, communication halts entirely (C, E).

49. Answer: A, B, D, E **Explanation:** SOA's documented limitations include limited scalability from resource sharing/coordination, increasing interdependencies, and (when using an ESB) a single point of failure — directly contradicting C.

50. Answer: B, C, D, E **Explanation:** Microservices evolved from SOA, addressing SOA's shortcomings by being fine-grained, favoring data duplication over sharing, and removing the centralized ESB dependency (consumers use lightweight APIs directly) — the opposite of retaining a centralized ESB (A is false).

51. Answer: A, D, E **Explanation:** Abstraction means clients see a service as a black box, understanding only what it does and how to use it via contracts/descriptions and its interface — not by having source code access (C is false), and this does not contradict interoperability (B is false).

52. Answer: B, C, D **Explanation:** SOA's benefits over monolithic architecture are faster time to market (reuse), efficient maintenance (small isolated services), and greater adaptability (modernization without full rewrites). Modifying a service should NOT significantly impact the overall process — that's the point of SOA (A is false), and reuse is a clear SOA advantage (E is false). ## Section H: Microservices Basics & Architecture (Q53-Q61)

53. Answer: A, C, D **Explanation:** The monolith-to-microservices shift moves from a shared database to database-per-service, from a tightly coupled unified app to loosely coupled services behind a gateway, and from all-or-nothing scaling/deployment to independent, per-service scaling and deployment — the reverse of B and E, which describe the monolithic side.

54. Answer: A, B, D, E **Explanation:** Key characteristics are decentralized architecture, scalability, failure resistance, and evolutionary capacity. A centralized shared database is a monolithic trait, not a microservices default (C is false — each microservice generally has its own database).

55. Answer: B, C, D **Explanation:** Failure resistance means an isolated failure only removes that specific functionality without crashing the whole app or cascading to other services — directly contradicting A and E.

56. Answer: A, B, D **Explanation:** Monoliths use a single team and a defined/unified tech stack (C, E are false), while microservices allow separate teams per service and diverse technology choices per service.

57. Answer: A, B, D, E **Explanation:** Documented problems solved include complex codebases, poor scalability, unacceptable downtime, and deficient tech stack capabilities. Microservices reduce complexity and improve testing per service, but do not “guarantee” elimination of all bugs (C is an overreach/false).

58. Answer: A, B, D, E **Explanation:** Each problem-solution pair matches except C — the actual solution to slow development/testing is autonomous teams *per service* (not one single team doing everything), directly opposing the microservices philosophy of decentralization.

59. Answer: C, D, E **Explanation:** The graph shows monolith time-to-market rising steeply with complexity, while microservices scale more gradually. At low complexity, monoliths are often faster to launch initially (which is why many companies start monolithic) — contradicting A and B.

60. Answer: A, C, D **Explanation:** The model explicitly shows separate databases per microservice (not shared — B is false) and independent scaling; microservices can use different languages/technologies (E is false), one of their core benefits.

61. Answer: A, C, D **Explanation:** Independent components (isolated impact), decentralized approach (no expensive centralized DB, applies varied technologies — E is false, it actually enables tech variety), and evolutionary capacity (fast feature growth) are correctly described. Failure resistance means the app keeps working minus that feature, not that it must go fully offline (B is false). ## Section I: Microservices Benefits & Adoption (Q62-Q69)

62. Answer: C **Explanation:** This is the textbook example of “Self-sufficiency” — each microservice/tool achieves a distinct business goal and can be maintained independently via API/HTTP integrations, not a monolithic pattern.

63. Answer: A, B, C, E **Explanation:** “Technical variety” explicitly allows different languages per service (Python for ML/AI, Java/GoLang for web/mobile/embedded, etc.) — the opposite of enforcing one unified stack (D is false, that's a monolithic trait).

64. Answer: A, D, E **Explanation:** This example is specifically cited under “Scalability” — showing independent, on-demand scaling per service and unlimited addition of new services (no hard limit — B is false).

65. Answer: D, E **Explanation:** This directly matches the “High Sustainability” benefit example given in the material — isolated failures don't cripple the whole app, and errors can be allocated/fixed without full outages.

66. Answer: A, C, E **Explanation:** Continuous Delivery lets teams update services without full-app downtime, and Easy Changes Implementation lets one service be modified/disabled temporarily without a full redeploy — CI/CD is actually easier with microservices (B, D are false).

67. Answer: A, B, C, E **Explanation:** The cited order is Data analytics (45%) > Database applications (41%) > CRM (38%) > Finance (34%) > HR (31%) — so Finance ranks ahead of HR, not the reverse (D is false).

68. Answer: A, B, D, E **Explanation:** Stats cited: 81.5% already use microservices, 92% reported successful switches, 84% of adopters plan to keep investing (a large majority, not “under 50%” — C is false), 87% say the expense was worth it, and only 1% won’t adopt.

69. Answer: A, C, D, E **Explanation:** Microservices fit companies expecting rapid growth, wanting varied technologies, able to staff multiple teams, and pursuing reliability/failure-proofing. A company satisfied with current speed and needing no scaling (B) has no strong driver to switch — this matches the exam paper’s own scenario logic (Q5, Q1 style). ## Section J: Microservices Use Cases (Q70–Q73)

70. Answer: A, C, E **Explanation:** Legacy/outdated applications are a named microservices use case, tackled by breaking the monolith into microservices and modernizing extracted parts incrementally (strangler-style) — not necessarily via a forced complete rewrite (D is false).

71. Answer: A, B, E **Explanation:** This matches the “Highly-Resilient Applications” use case explicitly citing healthcare/finance examples, where partial-functionality survival during failures is key. It’s not restricted to gaming (C is false), and real-time data processing can also be relevant to healthcare monitoring (D is an overreach/false in context).

72. Answer: A, C, E **Explanation:** Complex-logic apps (explicitly including social media and e-commerce platforms) are named as a foremost use case, benefiting from distinct features built with different technologies — not from monolithic simplicity (B is false), and the use case explicitly covers e-commerce too (D is false).

73. Answer: A, D, E **Explanation:** Data-heavy applications benefit from microservices reducing complexity from many dependencies; the related “Real-Time Data” use case is about on-demand resource allocation for changing data volumes — the two use cases are related, not unrelated (C is false), and microservices are recommended here, not avoided (B is false). ## Section K: Microservices Security (Q74–Q81)

74. Answer: A, B, C, E **Explanation:** Each pairing matches the documented pros/cons except D — JWT is explicitly called a “comprehensive option” that “senior developers should implement,” not the simplest, due to digital signature validation and the JWT-revocation problem.

75. Answer: A, C, D, E **Explanation:** Monolithic auth uses one DB/one token issuer, while microservices require a dedicated login service with distributed token verification and separate per-service databases — these architectures are explicitly different, not identical (B is false).

76. Answer: A, C, D, E **Explanation:** AWS Cognito, FusionAuth, and Auth0 are the three named CIAM platforms with their respective features as listed. UDDI is a service registry/discovery specification, unrelated to CIAM (B is false).

77. Answer: A, C, D **Explanation:** Best practices explicitly include HTTPS with certificate validation and service mesh usage (sidecar proxies) for simplifying communication, detecting errors, and encrypting/validating data — and a service mesh is explicitly said to streamline dev/test/deployment, not avoid doing so (E is false); relying on obscurity is never recommended (B is false).

78. Answer: A, B, D, E **Explanation:** All five best practices are: authentication/authorization, secured communication, secured data storage, sensitive data management, and API gateway+firewall. Sensitive data management explicitly recommends AVOIDING hardcoded credentials and following least-privilege principles — not hardcoding them (C is false).

79. Answer: B, C, D **Explanation:** The core challenges are decoupled architecture (different DBs/tech per service, complicating auth) and inter-service communication of sensitive data — requiring a fundamentally different, more complex approach than a monolith’s single shared database (A, E are false).

80. Answer: C, D, E **Explanation:** Robust security combines HTTPS + service mesh (secured communication), avoiding hardcoded credentials + API Gateway (sensitive data management/best practice), and least privilege + JWT (auth). Building everything custom from scratch (B) and defaulting to shared session storage regardless of trade-offs (A) are not presented as the most robust or recommended approaches.

81. Answer: A, B, D, E **Explanation:** Shared session storage trades some microservices principles for performance, and is documented as potentially becoming a bottleneck (contradicting C, which is false) — while JWT decentralizes info at the cost of larger network payloads. ## Section L: Microservices Migration (Q82–Q87)

82. Answer: B, C, D, E **Explanation:** Step 1 explicitly lists insufficient business analysis, poor team management, unoptimized development processes, insufficient team seniority, and lack of testing as causes to

rule out — a shared database is not listed here as an automatic root cause (A is false/overreach).

83. Answer: B, C, D **Explanation:** Steps 2–4 are: analyze if issues can be solved without converting (Step 2), optimize the monolith via DDD/Clean Architecture first (Step 3), then choose between complete vs. incremental rewrite (Step 4) — not skip straight to a big-bang approach (A, E are false).

84. Answer: A, B, C **Explanation:** Steps 5–7 are feature prioritization (with defined criteria — E is false), features extraction via a strangler application connected to (not replacing) the monolith (D is false), and implementing new features as microservices.

85. Answer: B, C, D **Explanation:** This is the textbook definition of a “Distributed Monolith” — services technically split apart but still tightly coupled/dependent, meaning original problems persist despite the architectural change. It is explicitly described as undesirable, not ideal (A, E are false).

86. Answer: A, B, D, E **Explanation:** The Distributed Monolith diagram shows widespread tangled dependencies (poor boundaries), while proper Microservices show more contained, clustered dependencies within logical groupings — the two are visually and conceptually distinct, not identical (C is false).

87. Answer: A, C, D **Explanation:** Careful problem diagnosis, thoughtful rewrite-approach selection, and disciplined strangler-pattern extraction all help avoid ending up with tightly coupled “distributed monolith” services. Skipping optimization (B) or ignoring prioritization criteria (E) increases the risk of a poor migration outcome. ## Section M: Real-World Microservices Use Cases (Q88–Q92)

88. Answer: B, C, D, E **Explanation:** Amazon’s response involved decomposing the monolith into single-purpose microservices connected via APIs, followed by CI/CD automation — the opposite of avoiding automation (A is false). Amazon is cited as using 1000+ microservices today.

89. Answer: B, C, D, E **Explanation:** Netflix faced database failures under growing load, prompting a move to AWS’s cloud (horizontal scaling) rather than staying on-premises (A is false), requiring decoupling the monolith — resulting in the now widely cited 700+ microservices figure.

90. Answer: A, B, C, E **Explanation:** Uber’s monolith handled all core functions in one codebase, where minor changes required full testing — the opposite of “trivial” fast releases (D is false, this was precisely the pain point) — leading to a shift to 500+ independently scalable microservices.

91. Answer: A, B, C, E **Explanation:** The stated figures are Amazon (1000+), Netflix (700+), Uber (500+); Netflix — not Uber — is explicitly called “the youngest business case for microservices adoption” in the material (D is false).

92. Answer: A, B, D, E **Explanation:** All three narratives share the same pattern: rapid growth exposing monolith limitations (scaling, deployment risk, merge/testing overhead), resolved through decomposition into independently scalable/deployable microservices. All three explicitly experienced problems (outages, merge conflicts, testing bottlenecks) pre-migration — contradicting C. ## Section N: Cloud Computing & AWS Case Study (Q93–Q100)

93. Answer: A, B, D, E **Explanation:** IaaS = infrastructure control; PaaS = no infrastructure management, focus on app deployment; SaaS = fully managed complete product (not requiring you to manage OS/infra — C is false, that description fits IaaS instead).

94. Answer: B, C, D, E **Explanation:** AWS benefits cited include most functionality, largest partner/customer community (APN with ISVs and integrators), fastest innovation pace (e.g., Lambda in 2014, SageMaker), and elasticity. AWS’s cost model is explicitly pay-as-you-go/variable, not “highest fixed cost” (A is false).

95. Answer: A, B, D **Explanation:** A company that builds/sells software is an ISV; when it formally partners with AWS for support/benefits, it becomes an APN Partner — these categories commonly overlap (many ISVs are also APN partners), contradicting C and E.

96. Answer: A, B, D, E **Explanation:** The paper (2003), S3 launch (2006), Netflix’s 2010 announcement, and the SQS/EC2 follow-up are all documented facts. The “excess capacity” story is explicitly called a “popular myth,” not a verified fact (C is false) — the real driver was standardizing infrastructure for faster internal development.

97. Answer: A, B, C, D **Explanation:** AWS (#1), Azure (#2, enterprise/hybrid strength), GCP (#3, TensorFlow/Kubernetes/AI strength), and IBM Cloud (RedHat/OpenShift advantage) are all accurately described. OCI is explicitly favored by *larger enterprises* and existing Oracle customers, not startups/SMBs (E is false).

98. Answer: A, B, D, E **Explanation:** The material explicitly cites AWS’s hybrid-cloud weakness (versus

Azure/IBM) and stresses that no market leadership is guaranteed long-term, requiring continuous innovation — directly contradicting the notion of “guaranteed permanent leadership” (C is false).

99. Answer: B, C, D, E **Explanation:** Netflix (7-year migration to microservices), Siemens (MindSphere IoT), Hotstar (5700 Gbps bandwidth, cost/time reduction), and BMW (CARASSO built in 6 months using named AWS services) all match documented facts. Adobe’s AWS use is explicitly tied to running its Creative Cloud Suite and multi-terabyte operating environments for customers — not merely marketing site hosting (A is false).

100. Answer: A, B, C, E **Explanation:** The Adobe case study explicitly discusses AWS Local Zones (proximity reduces latency), AWS Direct Connect (dedicated, more secure connectivity), a ~30ms optimal latency target, and persona-based resource demands. It explicitly states higher latency DOES negatively impact interactivity (playback, GUI responsiveness) — contradicting D.